# Fase 3 — Activación de la carga y lecturas del caudalímetro (Punto 3)

Rama: `fase-3-carga-tiempo-real` · Ejecución y auditoría: 27/10/2026 · Estado: **completa y verificada en la rama** (pendientes en la sección 11).

Este archivo tiene dos partes:

- **Parte 1:** informe técnico de lo implementado, lo verificado y la auditoría final.
- **Parte 2:** guía de estudio para la defensa oral (secciones A–J).

Convención de rutas: `SRC` = `src/main/java/ar/edu/iua/iw3`. La división del trabajo, los prompts y los bloques "Estado de la Parte N" están en `docs/fase-3-division-de-trabajo.md`.

---

# Parte 1 — Informe técnico

## 1. Alcance y resultado

La fase implementa el **Punto 3 de la consigna**. El **sistema de carga** (la isla donde se carga el camión) hace dos cosas:

1. **Se activa:** envía el número de orden y la contraseña de 5 dígitos que la balanza generó en la Fase 2. Si la orden existe, está en estado 2 y la contraseña es correcta, el backend responde el **preset** (kg a cargar) y registra `activationDateTime`. La orden **sigue en estado 2**.
2. **Envía lecturas** del caudalímetro (≈ 1 por segundo): masa acumulada, densidad, temperatura, caudal y estampa de tiempo (que asigna el servidor al recibir la lectura). El backend **descarta** las inválidas sin escribir nada y **guarda** las válidas en el histórico (`order_details`), actualizando en la **cabecera** de la orden los últimos valores válidos y las fechas de inicio y fin de carga.

**Resultado:** los 19 criterios de aceptación están cumplidos y verificados con pruebas ejecutadas (sección 7 y matriz de la sección 10). La auditoría encontró **un defecto menor** fuera de los criterios (H-14: una falla de base al confirmar la activación no respondía con `StandartResponse`), que se **corrigió y verificó** en un commit `fix` aparte, y varias limitaciones registradas. Además se implementó la **decisión 11** (`GET /api/v1/orders/{orderNumber}/details`, consulta paginada del histórico), aprobada el 27/10/2026. La consigna original en PDF **no estuvo disponible**: la comparación se hizo contra su resumen (`CLAUDE.md` §4, el plan y la tabla §2.1 de la división) y queda **no verificada contra el original**.

| # | Criterio de aceptación (división, sección 6) | Resultado |
|---|---|---|
| 1 | Activación correcta: 200 con `preset`, `activationDateTime` guardado, estado 2 | Cumplido |
| 2 | Activación repetida: 200, mismo preset, `activationDateTime` sin cambios | Cumplido |
| 3 | Contraseña incorrecta: 403, la orden no se activa y el mensaje no repite contraseñas | Cumplido |
| 4 | Inexistente 404 · fuera de estado 2 409 · campos ausentes 400 · sin token 401 | Cumplido |
| 5 | La activación no agrega registros al historial | Cumplido |
| 6 | Lectura válida: 200 `saved: true`, fila en `order_details` y cabecera actualizada | Cumplido |
| 7 | Inicio = estampa de la primera lectura válida (no cambia); fin = la de la última | Cumplido |
| 8 | Lectura inválida: 200 `saved: false` con motivo; no se guarda ni toca la cabecera | Cumplido |
| 9 | Masa igual a la anterior: válida | Cumplido |

| # | Criterio de aceptación (división, sección 6) | Resultado |
|---|---|---|
| 10 | Campo ausente, JSON malformado o valor no numérico: 400 | Cumplido |
| 11 | Lectura de orden no activada o fuera de estado 2: 409 · inexistente 404 · sin token 401 | Cumplido |
| 12 | Atomicidad: detalle y cabecera se guardan juntos o ninguno | Cumplido |
| 13 | `GET /orders/{n}` muestra los campos de carga, oculta la contraseña y no trae lista de detalles | Cumplido |
| 14 | Tablas y columnas nuevas sobre una base con órdenes existentes | Cumplido |
| 15 | Swagger documenta los dos endpoints con sus respuestas | Cumplido |
| 16 | Postman: carpeta 03 pasa y las 01 y 02 siguen pasando (46 aserciones) | Cumplido (160 aserciones, 0 fallas) |
| 17 | Tests unitarios y `./mvnw -B test` en verde | Cumplido (11 tests, 0 fallas) |
| 18 | 100 lecturas consecutivas sin degradación apreciable | Cumplido (79,6 ms promedio, sin degradación) |
| 19 | Informe, guía, documentación corregida, PDF y auditoría final | Cumplido con este documento |

## 2. Decisiones de diseño (D1 a D10)

Confirmadas por el equipo el 27/10/2026 (división, sección 3). En el plan quedan como **decisiones 18 a 27**.

| # (plan) | Tema | Decisión | Alternativa descartada y motivo |
|---|---|---|---|
| D1 (18) | Contraseña incorrecta | **403 Forbidden** con `ForbiddenException` nueva | 401: el cliente lo leería como un problema del token JWT, que sí es válido |
| D2 (19) | Activación repetida | **Idempotente**: 200, mismo preset, `activationDateTime` no cambia | 409 en la segunda: si el sistema de carga reintenta tras un corte de red, quedaría bloqueado |
| D3 (20) | Lectura descartada | **200** con `{ "saved": false, "reason": "..." }`; guardada: `{ "saved": true, "reason": null }` | 400 o 422: descartar no es un error del cliente; el caudalímetro sigue enviando y la consigna dice "descartar" |
| D4 (21) | Densidad fuera de (0, 1) | Se **descarta**; los **campos ausentes** dan **400** | Densidad inválida como 400: rompería la regla de "descartar" para un valor físico fuera de rango |
| D5 (22) | Masa igual a la anterior | **Válida**; "la anterior" es la **última masa válida** (cabecera) | Comparar con la última recibida: una lectura descartada contaminaría la comparación siguiente |
| D6 (23) | `timestamp` de la lectura | **Corregida tras la auditoría (F-13):** el servidor **siempre** usa su hora de recepción y el `timestamp` que mande el cliente se ignora | Antes era opcional con hora del cliente; la consigna dice «momento de recepción» y las estampas del cliente podían quedar desordenadas (H-20) |
| D7 (24) | Orden de comprobaciones | Activación: **400** → **404** → **409** → **403**. Lecturas: **404** → **400** → **409** → **descarte** | Otro orden: se revelaría el estado de una orden ante un pedido mal formado (activación) o se validaría el cuerpo de una orden que no existe (lecturas) |
| D8 (25) | Fin de carga | `loadingEndDateTime` es también la estampa de los últimos valores | Un campo `lastReadingDateTime` duplicado, siempre igual al fin de carga |
| D9 (26) | Historial de estados | La activación **no** agrega registros | Registrar la activación: no es un cambio de estado y la consigna pide registrar los cambios de estado |

| # (plan) | Tema | Decisión | Alternativa descartada y motivo |
|---|---|---|---|
| D10 (27) | Capa de negocio | **`IOrderDetailBusiness`** aparte; la activación en `IOrderBusiness` | Todo en `OrderBusiness`: la cátedra usa una interfaz de negocio por entidad |
| 11 (28) | Consulta del histórico | **`GET /api/v1/orders/{orderNumber}/details`** paginado: `page` desde 0, `size` 20 por defecto y máximo 100, orden fijo por `id` ascendente (se ignora el `sort` del cliente), respuesta `PagedModel`; 404 si la orden no existe | Sin consulta (solo `SELECT COUNT(*)`): no se podía ver el histórico por la API. Una lista dentro de `Order`: inflaría `GET /orders/{n}` |

La decisión 11 figuraba en la división como **no confirmada**; la aprobó Tomás Calvo el 27/10/2026 y se implementó en la Parte 3 (apartado G). En el plan queda como **decisión 28**.

## 3. Archivos de la fase

### 3.1 Inventario por parte

La **responsabilidad asignada** sale de la división del trabajo (sección 7). Los **cambios observados** salen de `git log main..fase-3-carga-tiempo-real` (autor de cada commit). Coinciden en todos los archivos salvo uno: el arreglo de H-14 en `OrderBusiness.activate` (archivo de la Parte 1) lo commiteó la Parte 3, con autorización de Tomás Calvo.

| Archivo | Acción | Motivo | Responsabilidad asignada | Cambios observados |
|---|---|---|---|---|
| `SRC/model/Order.java` | Modificado | 7 campos opcionales: `activationDateTime`, `loadingStartDateTime`, `loadingEndDateTime`, `lastAccumulatedMass`, `lastDensity`, `lastTemperature`, `lastFlowRate` | Parte 1 · Martín González | `edceb00` (Martin Gonzalez) |
| `SRC/model/OrderDetail.java` | Creado | Entidad del histórico (`order_details`) | Parte 1 | `0ff6542` (Martin Gonzalez) |
| `SRC/model/persistence/OrderDetailRepository.java` | Creado | `JpaRepository<OrderDetail, Long>` sin métodos propios | Parte 1 | `0ff6542` |
| `SRC/business/exception/ForbiddenException.java` | Creado | Excepción *checked* → 403 | Parte 1 | `f6b8576` |
| `SRC/controllers/Constants.java` | Modificado | `URL_INTEGRATION_LOADER` | Parte 1 | `a1c3eb2` |
| `SRC/business/IOrderBusiness.java`, `OrderBusiness.java` | Modificados | Método `activate` | Parte 1 | `cca066e` |
| `SRC/integration/loader/model/ActivationRequest.java`, `ActivationResponse.java` | Creados | DTO de la activación | Parte 1 | `61f65d6` |
| `SRC/integration/loader/controllers/OrderLoaderRestController.java` | Creado | Endpoint `POST /activation` | Parte 1 | `b090dfa` |
| `SRC/business/OrderDetailValidator.java` | Creado | Reglas de descarte (función pura) | Parte 2 · Joaquín Contreras | `0e84cb8` (joacoContreras) |

| Archivo | Acción | Motivo | Responsabilidad asignada | Cambios observados |
|---|---|---|---|---|
| `src/test/java/ar/edu/iua/iw3/business/OrderDetailValidatorTest.java` | Creado | 8 tests unitarios del descarte | Parte 2 | `d5ed889` |
| `SRC/business/DetailResult.java` | Creado | Resultado de una lectura ( `saved` , `reason` ) | Parte 2 | `6eb915c` |
| `SRC/business/IOrderDetailBusiness.java` | Creado | Interfaz con `add` | Parte 2 | `f324515` |
| `SRC/business/OrderDetailBusiness.java` | Creado | Recepción, descarte, histórico y cabecera | Parte 2 | `85fc7cf` y `5971075` ( `saveAndFlush` ) |
| `SRC/integration/loader/model/DetailRequest.java` , `DetailResponse.java` | Creados | DTO de las lecturas | Parte 2 | `7540a79` |
| `OrderLoaderRestController.java` | Modificado | **Agrega** `POST /orders/{orderNumber}/details` sin tocar la activación | Parte 2 | `3861b3e` |
| `SRC/business/OrderBusiness.java` | Modificado | `activate` usa `saveAndFlush` (arreglo de H-14) | Parte 1 (dueño del archivo); el arreglo lo hizo la Parte 3 con autorización de Tomás Calvo | `4ed54c8` (tomas-calvo01) |
| `SRC/model/persistence/OrderDetailRepository.java` | Modificado | `findByOrder(Order, Pageable)` con orden fijo por `id` (decisión 11) | Parte 3 (apartado G) | `9cc4a05` |
| `SRC/business/IOrderDetailBusiness.java` , `OrderDetailBusiness.java` | Modificados | Método `list` (decisión 11) | Parte 3 (apartado G) | `935d11e` |
| `SRC/controllers/OrderRestController.java` | Modificado | `GET /orders/{orderNumber}/details` paginado (decisión 11) | Parte 3 (apartado G) | `191e327` |
| `docs/postman/IW3-Gas-Terminal.postman_collection.json` | Modificado | Carpeta 03 (44 pedidos, 114 aserciones) y 6 variables propias | Parte 3 · Tomás Calvo | `f07627a` y `041308a` (tomas-calvo01) |
| `docs/fase-3-carga-tiempo-real.md` y su PDF | Creados | Este informe y su guía | Parte 3 | Ídem |
| `CLAUDE.md` , `docs/plan-de-trabajo.md` , `docs/guia-estudio-src.md` , `docs/fase-2-pesaje-inicial.md` , `docs/pdf/` | Modificados | Correcciones del apartado E y PDF regenerados | Parte 3 | Ídem |
| `docs/fase-3-division-de-trabajo.md` | Modificado | Bloques "Estado de la Parte N" | Cada parte | `98357e5` , `8dd796e` (Parte 1), `4679852` (Parte 2), Parte 3 |

Benjamín Alasino no programa en esta fase: revisa y aprueba el PR.

### 3.2 Tablas y columnas

| Tabla | Cambio | Detalle |
|---|---|---|
| `orders` | 7 columnas nuevas, todas nulables | `activation_date_time`, `loading_start_date_time`, `loading_end_date_time` (`datetime(6)`); `last_accumulated_mass`, `last_density`, `last_temperature`, `last_flow_rate` (`double`) |
| `order_details` | Nueva | `id` (auto), `accumulated_mass`, `density`, `flow_rate`, `temperature` (`double NOT NULL`), `reading_timestamp` (`datetime(6) NOT NULL`), `id_order` (FK a `orders.id`, `NOT NULL`) |

## 4. Recorridos de las peticiones

### 4.1 `POST /api/v1/integration/loader/activation`

```
Sistema de carga ──POST /api/v1/integration/loader/activation {"orderNumber":120503639,"password":36797}──▶
   │
   ├─ 1. JWTAuthorizationFilter: valida el token Bearer (sin token → 401)
   ├─ 2. Jackson: deserializa el cuerpo en ActivationRequest (JSON malformado o "abc" → 400)
   ├─ 3. OrderLoaderRestController.activate → orderBusiness.activate(orderNumber, password)
   │
   ├─ 4. OrderBusiness.activate (@Transactional(rollbackFor = Exception.class)), FUERA del try:
   │      a. orderNumber o password nulos              → 400 BadRequestException
   │      b. load(orderNumber)                         → 404 NotFoundException
   │      c. status != INITIAL_WEIGHING_REGISTERED (2) → 409 ConflictException
   │      d. !password.equals(activationPassword)       → 403 ForbiddenException
   │      DENTRO del try (solo si activationDateTime es nulo):
   │      e. activationDateTime = LocalDateTime.now(); orderDAO.saveAndFlush(order)   → UPDATE orders
   │      (si ya estaba activada no escribe nada: idempotente; no escribe en order_status_logs)
   │
   ├─ 5. Controller: new ActivationResponse(order.getPreset())
   ▼
◀── 200 OK {"preset": 15000.0}
```

### 4.2 `POST /api/v1/integration/loader/orders/{orderNumber}/details`

```
Caudalímetro ──POST .../loader/orders/120503639/details {"accumulatedMass":250.0,"density":0.52,
               "temperature":19.5,"flowRate":3600.0}──▶
   │
   ├─ 1. JWTAuthorizationFilter (sin token → 401) · Jackson → DetailRequest (malformado / "abc" → 400)
   ├─ 2. OrderLoaderRestController.addDetail → orderDetailBusiness.add(orderNumber, masa, densidad, temp, caudal,
ts)
   │
   ├─ 3. OrderDetailBusiness.add (@Transactional(rollbackFor = Exception.class)), FUERA del try:
   │      a. orderBusiness.load(orderNumber)                              → 404
   │      b. algún Double nulo                                            → 400
   │      c. status != 2  o  activationDateTime == null                   → 409
   │      d. OrderDetailValidator.discardReasons(masa, densidad, caudal, order.lastAccumulatedMass)
   │         si hay motivos → return DetailResult.discarded("motivo1; motivo2")   (no escribe nada)
   │      DENTRO del try:
   │      e. estampa = LocalDateTime.now() (hora de recepción del servidor)
   │      f. orderDetailDAO.save(new OrderDetail(...))                     → INSERT order_details
   │      g. cabecera: last* = valores; loadingStartDateTime si es nulo; loadingEndDateTime = estampa
   │      h. orderDAO.saveAndFlush(order)                                  → UPDATE orders (dentro del try)
   │      i. return DetailResult.saved()
   │
   ├─ 4. Controller: new DetailResponse(result.isSaved(), result.getReason())
   ▼
◀── 200 OK {"saved": true, "reason": null}     ó     200 OK {"saved": false, "reason": "..."}
```

Si algo falla dentro del `try` (por ejemplo, la base rechaza el `UPDATE`), se loguea, se lanza `BusinessException`, la transacción se revierte completa (ni detalle ni cabecera) y el controller responde 500 con `StandartResponse`. Desde el arreglo de H-14 la activación se comporta igual.

### 4.3 `GET /api/v1/orders/{orderNumber}/details?page=&size=` (decisión 11)

```
Cliente ──GET /api/v1/orders/120503639/details?page=0&size=2──►
   │
   ├─ 1. JWTAuthorizationFilter (sin token → 401)
   ├─ 2. OrderRestController.details: size = min(size, 100); PageRequest.of(page, size) (sin el sort del cliente)
   ├─ 3. OrderDetailBusiness.list: orderBusiness.load(orderNumber) → 404 (fuera del try)
   │      DENTRO del try: orderDetailDAO.findByOrder(order, pageable)
   │      → select ... from order_details where id_order = ? order by id asc limit ?, ?  (+ count)
   ├─ 4. Controller: new PagedModel<>(page)
   ▼
◄── 200 OK {"content": [ {"id":..,"accumulatedMass":100.0,"density":0.5,"temperature":20.0,"flowRate":3000.0,
            "timestamp":"2026-10-28T10:00:01"}, ... ], "page":
{"size":2,"number":0,"totalElements":4,"totalPages":2}}
```

Cada lectura se publica sin la orden (`@JsonIgnore` en `OrderDetail.order`).

## 5. Funciones y métodos

### `OrderBusiness.activate` — `SRC/business/OrderBusiness.java`

- **Firma:** `Order activate(Integer orderNumber, Integer password) throws NotFoundException, BadRequestException, ConflictException, ForbiddenException, BusinessException`.
- **Transacción:** `@Transactional(rollbackFor = Exception.class)`.
- **Validaciones (fuera del `try`, en este orden):** nulos → 400 `"orderNumber y password son obligatorios"`; `load` → 404 `"No se encuentra la Orden numero=<n>"`; estado ≠ 2 → 409 `"La orden <n> no está lista para activar (estado <e>)"`; contraseña distinta → 403 `"La contraseña de activación es incorrecta"`.
- **Efecto:** si `activationDateTime` es nulo, lo fija con `LocalDateTime.now()` y llama a `orderDAO.saveAndFlush(order)` (hasta el arreglo de H-14 era `save`). No cambia el estado ni escribe historial.
- **Retorno:** la orden (el controller toma el `preset`).
- **Dependencias:** `OrderRepository` (`orderDAO`).

### `OrderDetailBusiness.add` — `SRC/business/OrderDetailBusiness.java`

- **Firma:** `DetailResult add(Integer orderNumber, Double accumulatedMass, Double density, Double temperature, Double flowRate) throws NotFoundException, BadRequestException, ConflictException, BusinessException`.
- **Dependencias:** `IOrderBusiness` (para `load`, sin duplicar la búsqueda), `OrderRepository`, `OrderDetailRepository` y `OrderDetailValidator`.
- **Validaciones (fuera del `try`):** 404 → 400 `"accumulatedMass, density, temperature y flowRate son obligatorios"` → 409 `"La orden <n> no acepta lecturas: debe estar en estado 2 y activada (estado <e>, activada=<true/false>)"` → descarte.
- **Efecto (lectura válida):** inserta `OrderDetail`, actualiza los cuatro últimos valores, `loadingStartDateTime` (solo si es nulo) y `loadingEndDateTime` (siempre), y hace `saveAndFlush` de la orden.
- **Retorno:** `DetailResult.saved()` o `DetailResult.discarded(motivos unidos con "; ")`.

### `OrderDetailBusiness.list` — `SRC/business/OrderDetailBusiness.java` (decisión 11)

- **Firma:** `Page<OrderDetail> list(Integer orderNumber, Pageable pageable) throws NotFoundException, BusinessException`.
- **Validación:** `orderBusiness.load(orderNumber)` fuera del `try` → 404.
- **Efecto:** solo lectura; `orderDetailDAO.findByOrder(order, pageable)`. Una falla técnica → `BusinessException` (500).

### `OrderDetailRepository.findByOrder` — `SRC/model/persistence/OrderDetailRepository.java`

- `@Query("select d from OrderDetail d where d.order = ?1 order by d.id asc") Page<OrderDetail> findByOrder(Order order, Pageable pageable)`. El orden está fijo en la consulta, como `OrderStatusLogRepository.findByOrder`.

### `OrderRestController.details` — `SRC/controllers/OrderRestController.java` (decisión 11)

- `GET /{orderNumber}/details` con `@ParameterObject @PageableDefault(size = 20) Pageable`. Acota `size` a `MAX_PAGE_SIZE` (100), arma `PageRequest.of(page, size)` sin el `sort` del cliente y responde `new PagedModel<>(...)`. Traduce 404 y 500. Swagger: `@Operation` y `@ApiResponse` 200 y 404 (Swagger muestra también `sort` porque viene de `Pageable`, igual que en `GET /orders`; se ignora).

### `OrderDetailValidator.discardReasons` — `SRC/business/OrderDetailValidator.java`

- **Firma:** `static List<String> discardReasons(double accumulatedMass, double density, double flowRate, Double previousMass)`. Clase `final` de paquete, constructor privado, función pura.
- **Reglas (se evalúan todas):** `!(flowRate > 0)` → `"el caudal (flowRate) debe ser mayor que 0"`; `!(accumulatedMass > 0)` → `"la masa acumulada (accumulatedMass) debe ser mayor que 0"`; `previousMass != null && accumulatedMass < previousMass` → `"la masa acumulada es menor que la anterior (<valor>)"`; `!(density > 0 && density < 1)` → `"la densidad (density) debe estar entre 0 y 1, sin incluirlos"`. La forma `!(x > 0)` descarta también `NaN`. La temperatura no tiene regla.

### `DetailResult` — `SRC/business/DetailResult.java`

- Clase pública inmutable con `saved` (boolean) y `reason` (String), getters de Lombok, constructor privado y fábricas `saved()` y `discarded(reason)`.

### `OrderLoaderRestController` — `SRC/integration/loader/controllers/OrderLoaderRestController.java`

- `@RestController`, `@RequestMapping(Constants.URL_INTEGRATION_LOADER)`, `@Tag("Integración sistema de carga")`, extiende `BaseRestController`. Inyecta `IOrderBusiness`, `IOrderDetailBusiness` e `IStandartResponseBusiness`.
- `activate(@RequestBody ActivationRequest)`: 200 `ActivationResponse`; traduce 400, 404, 409, 403 y 500.
- `addDetail(@PathVariable Integer orderNumber, @RequestBody DetailRequest)`: 200 `DetailResponse`; traduce 400, 404, 409 y 500.
- Swagger: `@Operation` y `@ApiResponse` 200/400/403/404/409 (activación) y 200/400/404/409 (lecturas).

### DTO — `SRC/integration/loader/model/`

- `ActivationRequest` (`Integer orderNumber`, `Integer password`), `ActivationResponse` (`Double preset`), `DetailRequest` (`Double accumulatedMass, density, temperature, flowRate`, `LocalDateTime timestamp`), `DetailResponse` (`boolean saved`, `String reason`). Se usan objetos (`Double`, `Integer`) para que un campo ausente llegue como `null` y dé 400 (D4).

### Entidades

- `OrderDetail`: `@ManyToOne(optional = false, fetch = LAZY)` a `Order` con `@JsonIgnore` y columna `id_order`; cuatro `double` `nullable = false`; `timestamp` en la columna `reading_timestamp`.
- `Order`: los 7 campos de la sección 3.2, `@Schema(accessMode = READ_ONLY)`. **No** tiene `@OneToMany` a los detalles.

### `ForbiddenException` — `SRC/business/exception/ForbiddenException.java`

- *Checked*, mismo molde que `ConflictException` (`@NoArgsConstructor` y constructor con `@Builder`).

## 6. Cambios de comportamiento respecto de la Fase 2

| Escenario | Fase 2 | Fase 3 |
|---|---|---|
| `POST /api/v1/integration/loader/activation` | No existía | 200 con el preset; 400, 401, 403, 404, 409 |
| `POST /api/v1/integration/loader/orders/{n}/details` | No existía | 200 guardada o descartada; 400, 401, 404, 409 |
| `GET /api/v1/orders/{n}/details?page=&size=` | No existía | 200 con la página de lecturas guardadas (`PagedModel`); 401, 404 |
| `GET /api/v1/orders/{n}` | Hasta `initialWeighingDateTime` | Suma los 7 campos de carga (en `null` hasta que existen); sigue sin `activationPassword` y sin lista de detalles |
| Orden en estado 2 | Solo esperaba la Fase 3 | Puede activarse y, ya activada, recibir lecturas. Sigue en estado 2 |
| Historial `/status-log` | 2 registros tras la tara | Sigue con 2 registros: la activación no es un cambio de estado |
| Esquema de la base | `orders` con columnas de tara | `orders` + 7 columnas y la tabla `order_details` |
| Colección Postman | 21 pedidos, 46 aserciones | 65 pedidos, 160 aserciones |
| Tests de Maven | 3 | 11 (suma `OrderDetailValidatorTest`) |

## 7. Pruebas y verificación

Todas ejecutadas el **27/10/2026** contra MySQL real (contenedor `iw3-gas-mysql`, base `iw3_db`), con la app en `http://localhost:8080` (perfil por defecto).

### 7.1 Esquema sobre una base con órdenes (criterio 14)

La base local estaba con el esquema de la **Fase 1** y una orden (`500181085`, estado 1). Al arrancar la app (`Started Iw3Application in 11.984 seconds`, sin `ERROR`), Hibernate agregó las 10 columnas que faltaban en `orders` (3 de la Fase 2 y 7 de la Fase 3), **todas nulables**, y creó `order_details` con su clave foránea. La orden previa quedó intacta, con los campos nuevos en `NULL`.

### 7.2 Tests de Maven (criterio 17)

Comando: `./mvnw -B test` (con la base arriba).

| Clase | Tests | Resultado |
|---|---|---|
| `ActivationPasswordGeneratorTest` | `generatesFiveDigitPasswords`, `doesNotAlwaysReturnTheSameValue` | 2/2 |

| Clase | Tests | Resultado |
|---|---|---|
| `OrderDetailValidatorTest` | `discardReasons_validReadings_returnsEmptyList`, `discardReasons_equalMass_isValid`, `discardReasons_flowRateZeroOrNegative_discards`, `discardReasons_accumulatedMassZeroOrNegative_discards`, `discardReasons_massLessThanPrevious_discards`, `discardReasons_densityOutOfRange_discards`, `discardReasons_nanValues_discards`, `discardReasons_multipleInvalidConditions_accumulatesAllReasons` | 8/8 |
| `Iw3ApplicationTests` | `contextLoads` | 1/1 |

**Resultado:** `Tests run: 11, Failures: 0, Errors: 0, Skipped: 0` · `BUILD SUCCESS`.

## 7.3 Postman con newman (criterio 16)

Comando: `npx newman@6 run docs/postman/IW3-Gas-Terminal.postman_collection.json`. Primero se corrió la carpeta 03 sin los pedidos de la decisión 11 (38 pedidos): dos corridas con **59 pedidos y 146 aserciones, 0 fallas**. Después de implementar la decisión 11 se agregaron 6 pedidos y se corrió dos veces más:

| Carpeta | Pedidos | Aserciones | Fallas |
|---|---|---|---|
| 00 - Autenticación | 2 | 4 | 0 |
| 01 - Fase 1 (SAP) | 8 | 15 | 0 |
| 02 - Fase 2 (TMS) | 11 | 27 | 0 |
| **03 - Fase 3 (sistema de carga)** | **44** | **114** | **0** |
| **Total** | **65** | **160** | **0** |

Las carpetas 00 a 02 mantienen sus **46 aserciones** sin cambios (los dos `git diff` de la colección, 1910 y 263 líneas, solo agregan). Variables propias de la carpeta 03: `orderNumberLoad`, `orderNumberLoadPending`, `orderNumberLoadInactive`, `activationPasswordLoad`, `wrongPasswordLoad` y `activationDateTimeLoad`.

| Grupo | Pedido | Esperado | Resultado |
|---|---|---|---|
| Preparación | Crear orden para la prueba de carga | 201 + `location` | Aprobado |
| Preparación | Tara de la orden de carga | 200, contraseña de 5 dígitos guardada | Aprobado |
| Preparación | Crear orden que queda en estado 1 | 201 | Aprobado |
| Preparación | Crear orden con tara sin activar · Tara de la orden sin activar | 201 · 200 | Aprobado |
| Activación | Activación sin campos · sin password | 400, `son obligatorios` | Aprobado |
| Activación | Activación de orden inexistente | 404, `No se encuentra la Orden` | Aprobado |
| Activación | Activación de orden en estado 1 | 409, `no está lista para activar`, `estado 1` | Aprobado |
| Activación | Activación con contraseña incorrecta | 403, `contraseña de activación es incorrecta`; el cuerpo no contiene la contraseña enviada ni la correcta | Aprobado |
| Activación | Activación con JSON malformado · sin token | 400 · 401 | Aprobado |
| Activación | Activación correcta | 200, `preset` 15000 | Aprobado |
| Activación | Consultar la orden activada | estado 2, `activationDateTime` presente, sin `activationPassword`, campos de carga en `null`, sin lista de detalles | Aprobado |
| Activación | Activar de nuevo · Consultar tras activar de nuevo | 200 con el mismo preset · misma `activationDateTime` | Aprobado |
| Activación | Historial de estados tras activar | 2 registros (`null→1`, `1→2`) | Aprobado |

| Grupo | Pedido | Esperado | Resultado |
|---|---|---|---|
| Lecturas válidas | Lectura 1 (100 kg, `10:00:01`) · Lectura 2 (250 kg, `10:00:06`) | `saved: true`, `reason: null` | Aprobado |
| Lecturas válidas | Consultar la cabecera tras dos lecturas | últimos valores de la 2; inicio `2026-10-28T10:00:01`; fin `2026-10-28T10:00:06` | Aprobado |
| Descartes | Masa menor (200) · caudal 0 · caudal −5 · masa 0 · densidad 0 · 1 · 1.5 | `saved: false` con el fragmento del motivo | Aprobado (7) |
| Descartes | Consultar la cabecera tras los descartes | sin cambios | Aprobado |
| Límite | Masa igual a la anterior (250) · cuarta lectura (400 kg) con un `timestamp` de 2020 enviado a propósito | `saved: true`; el `timestamp` del cliente se ignora | Aprobado |
| Límite | Consultar la cabecera tras los casos límite | últimos valores = 400 kg; inicio sin cambios; fin = hora del servidor | Aprobado |
| Histórico (decisión 11) | Lecturas guardadas: primera página (`page=0&size=2&sort=id,desc`) | 200; `totalElements` 4, `totalPages` 2; lecturas 1 y 2 por `id` ascendente (se ignora el `sort`); cada lectura sin la orden | Aprobado |
| Histórico | Segunda página (`page=1&size=2`) | masa igual (250) y cuarta lectura (400); ninguna estampa es de 2020; los descartes no aparecen | Aprobado |
| Histórico | `size=500` · orden sin activar | `size` acotado a 100 con las 4 lecturas · página vacía | Aprobado |
| Histórico | Orden inexistente · sin token | 404 `No se encuentra la Orden` · 401 | Aprobado |
| Errores | Sin un campo · JSON malformado · valor no numérico | 400 | Aprobado |
| Errores | Orden no activada · orden en estado 1 | 409 (`activada=false` / `estado 1`) | Aprobado |
| Errores | Orden inexistente · sin token | 404 · 401 | Aprobado |

## 7.4 Cruce con la base (criterios 6, 8 y 11)

Para la orden de la primera corrida (`120288201`, `id = 6`):

```
SELECT COUNT(*) FROM order_details WHERE id_order = 6;   -- 4
```

Las 4 filas son las lecturas guardadas: 100 (`10:00:01`), 250 (`10:00:06`), 250 (`10:00:21`) y 400 (hora del servidor). Las 7 descartadas no están. La orden en estado 1 y la orden sin activar tienen **0** detalles. En la segunda corrida (orden `120503639`) el resultado fue el mismo: 4 detalles y 0 en la orden sin activar.

## 7.5 Pruebas de auditoría fuera de Postman

Script temporal de auditoría (en el *scratchpad* de la sesión, fuera del repo) contra la app: **18 PASS, 0 FAIL**, más las pruebas sobre la base local de 7.6 y 7.7.

| Comprobación | Pedido | Resultado |
|---|---|---|
| D7 activación: 400 antes que 404 | `{"orderNumber": 999999999}` | 400 `orderNumber y password son obligatorios` |
| D7 activación: 409 antes que 403 | orden en estado 1 con contraseña 11111 | 409 `... no está lista para activar (estado 1)` |
| D7 lecturas: 404 antes que 400 | `{}` a la orden 999999999 | 404 |
| D7 lecturas: 400 antes que 409 | `{}` a una orden en estado 1 | 400 |
| D7 lecturas: 409 antes que el descarte | caudal 0 a una orden sin activar | 409 (`activada=false`) |

| Comprobación | Pedido | Resultado |
|---|---|---|
| D1: el 403 no activa | contraseña incorrecta a una orden sin activar y `GET` | 403; `activationDateTime` sigue en `null` |
| D1: el 403 no altera una orden activada | contraseña incorrecta a una orden activada | 403; `activationDateTime` sin cambios |
| Temperatura negativa | −8,5 °C con el resto válido | `saved: true` |
| `timestamp` con formato inválido | `"28/10/2026 11:00"` | 400 |
| Swagger (criterio 15) | `GET /v3/api-docs` | activación con 200/400/403/404/409; lecturas con 200/400/404/409; esquema `Order` con los 7 campos y sin `activationPassword`; esquemas de los 4 DTO |
| Eficiencia (criterio 18) | 100 lecturas seguidas, masa creciente | 100/100 guardadas; **7,96 s** en total; **79,6 ms** promedio; primeras 10: 84,6 ms; últimas 10: 80,4 ms (sin degradación) |
| Contraseña en logs | búsqueda de las contraseñas de la corrida en el log de la app (≈ 8650 líneas al momento) | 0 apariciones; el SQL se loguea con `?` |

La Parte 2 había medido **36,1 ms** por lectura en su máquina; la diferencia es del equipo, y en ningún caso hay degradación entre las primeras y las últimas lecturas.

### 7.6 Preparación para la Fase 4 (estado 3)

Sobre la orden de auditoría (`120598199`, activada y con 103 lecturas), en la base **local**:

```
UPDATE orders SET status = 3 WHERE order_number = 120598199;
```

- Lectura válida → **409** `La orden 120598199 no acepta lecturas: debe estar en estado 2 y activada (estado 3, activada=true)`.
- Activación → **409** `La orden 120598199 no está lista para activar (estado 3)`.
- `order_details` siguió con 103 filas y la cabecera no cambió. Después se volvió a `status = 2`.

Conclusión: cuando la Fase 4 pase la orden a estado 3, las lecturas quedarán rechazadas **sin modificar código de la Fase 3**.

### 7.7 Atomicidad (criterio 12) y falla al confirmar (trampa 12)

**Lecturas.** Se agregó una restricción temporal `CHECK (last_accumulated_mass IS NULL OR last_accumulated_mass < 5000)` en `orders` y se envió una lectura válida de 6000 kg: el `INSERT` del detalle pasa, pero el `UPDATE` de la cabecera falla.

- Respuesta: **500 con** `StandartResponse` (`message: null`, `code: 500`, `devInfo` con la traza de `BusinessException`).
- `order_details` siguió con 103 filas (ninguna de 6000 kg) y la cabecera no cambió: **se revirtió todo**.
- Se quitó la restricción y la misma lectura respondió `saved: true`.

**Activación.** Se creó un *trigger* temporal que rechaza el `UPDATE` que pone `activation_date_time` y se activó una orden nueva con la contraseña correcta.

- Respuesta: **500 con el formato por defecto de Spring** (`timestamp`, `status`, `error`, `trace`, `message`, `path`), **no** `StandartResponse` → hallazgo **H-14**.
- `activation_date_time` siguió en `NULL` (la transacción se revirtió).
- La respuesta (22 KB) y el log **no** contienen la contraseña (el SQL de la traza muestra `activation_password=?`).
- Se borró el *trigger* y la activación respondió 200.

**Activación después del arreglo de H-14** (`4ed54c8`, `saveAndFlush`). Se repitió la misma prueba con una orden nueva (`121698543`):

- Respuesta: **500 con `StandartResponse`** (`message`, `code: 500`, `devInfo` con la traza de `BusinessException`).
- `activation_date_time` siguió en `NULL` y la respuesta no contiene la contraseña.
- Sin el *trigger*, la activación respondió 200 con el preset.

### 7.8 Regresiones de las Fases 1 y 2

- newman: las 46 aserciones de las carpetas 00 a 02 pasan (antes y después de agregar la carpeta 03, y después del arreglo de H-14 y de la decisión 11).
- `ActivationPasswordGeneratorTest` y `contextLoads` en verde (`./mvnw -B test`: 11 tests, 0 fallas, también después de los cambios de código).
- `pom.xml` sin cambios respecto de `main` (sin dependencias nuevas).

## 8. Auditoría del código

| Revisión | Resultado |
|---|---|
| D1 a D10 | El código hace exactamente lo decidido (detalle en la matriz de la sección 10) |
| Orden de comprobaciones y `try` | `activate`: 400 → 404 → 409 → 403, todas antes del `try`. `add`: 404 → 400 → 409 → descarte, todas antes del `try`. Verificado también con pedidos que combinan dos errores (7.5) |
| Lógica duplicada | No. `OrderDetailBusiness` reutiliza `IOrderBusiness.load`; el descarte vive solo en `OrderDetailValidator` |
| Código de las Fases 4 o 5 | No hay (búsqueda de `close`, `finalWeight`, `reconcil`, `CLOSED_FOR_LOADING`, `FINISHED`, `AVG` en `src/main`: sin resultados fuera de `OrderStatus`, que tiene los 4 estados desde la Fase 1) |
| Dependencias nuevas | No (`git diff main -- pom.xml` vacío) |
| Lista de detalles en `Order` | No. La única `@OneToMany` del modelo es la de `Truck` → `TruckTank` (Fase 1) |
| Contraseña en mensajes | No: el 403 es un texto fijo; el 400 menciona la palabra "password", no un valor |
| Contraseña en logs | No: 0 apariciones en el log; el SQL usa `?` |
| Contraseña en consultas | No: `@JsonIgnore` en `Order.activationPassword` y fuera del esquema de Swagger |
| Commits de la rama | Todos de ≤ 100 líneas salvo las excepciones permitidas: `1578e5c` (documento nuevo completo) y los dos del JSON de Postman (`f07627a`, `041308a`) |
| Consigna original en PDF | **No verificado contra el original**: el PDF no está en el repo ni en las máquinas del equipo. Se contrastó contra el resumen de `CLAUDE.md` §4, el plan y la división §2.1 |

## 9. Hallazgos y limitaciones

La numeración continúa la de la Fase 2 (H-12 y H-13).

| ID | Tipo | Detalle | Evidencia | Qué se hace |
|---|---|---|---|---|
| H-14 | **Defecto menor — corregido** | Una falla de base al **confirmar** la activación respondía 500 con el formato por defecto de Spring, no con `StandartResponse`. `OrderBusiness.activate` usaba `orderDAO.save(order)`: el `UPDATE` se enviaba al confirmar la transacción, después del `try/catch`. La Parte 2 había resuelto lo mismo en las lecturas con `saveAndFlush` | Sección 7.7: `POST /api/v1/integration/loader/activation` con la contraseña correcta y un *trigger* que rechaza el `UPDATE` → `500 {timestamp, status, error, trace, message, path}`; la orden no quedaba activada y no se filtraba la contraseña | Archivo de la **Parte 1 (Martín González)**: `SRC/business/OrderBusiness.java`, método `activate`. **Corregido** en `4ed54c8` (`fix(business)`): `orderDAO.saveAndFlush(order)`, autorizado por Tomás Calvo el 27/10/2026. Verificado: la misma prueba ahora responde 500 con `StandartResponse`. `registerTare` (Fase 2) tiene el mismo patrón; queda fuera del alcance de esta fase |
| H-15 | **Resuelto tras la auditoría (F-01)** | Dos lecturas simultáneas de la misma orden hacían que MySQL abortara una con *Deadlock found when trying to get lock* (500), y podían leer la misma masa anterior. Ahora la orden se busca con `PESSIMISTIC_WRITE` (`findByOrderNumberForUpdate`) y las lecturas de una orden se procesan de a una | Auditoría del PR #7; verificado con 150 lecturas simultáneas: 0 respuestas 500 y 0 deadlocks | Corregido; las activaciones simultáneas siguen siendo H-16 |
| H-16 | Limitación | Dos activaciones simultáneas de una orden sin activar podrían escribir `activationDateTime` dos veces (gana la última) | Análisis de código (Parte 1) | Aceptado para el parcial |
| H-17 | Limitación de seguridad | Contraseña de 5 dígitos (90.000 combinaciones) sin límite de intentos | Diseño de la Fase 2 | Etapa de seguridad del final |
| H-18 | Limitación | El token vence en 1 h: una carga más larga exige volver a loguearse | `AuthConstants` | Etapa de seguridad del final |
| H-19 | **Resuelto tras la auditoría (F-13)** | Antes un `timestamp` con `Z` se aceptaba y la zona se descartaba sin aviso. Ahora el servidor ignora cualquier `timestamp` del cliente (un valor con formato inválido ya no produce 400 por ese campo) | Sección 7.5 (script de auditoría) | Corregido al quitar el campo de `DetailRequest` |
| H-20 | **Resuelto tras la auditoría (F-13)** | Mezclar estampas del cliente y del servidor dejaba el fin de carga antes que el inicio. Con la hora de recepción siempre del servidor, las estampas son crecientes | `SELECT` sobre la orden 120288201 | Corregido |
| H-21 | Observación | Jackson acepta números enviados como texto: la contraseña correcta como texto (`"password": "<5 dígitos>"`) activa y `"accumulatedMass": "50"` se guarda. Un texto no numérico (`"abc"`) sí da 400 | Sección 7.5 | Comportamiento por defecto de Jackson; no se cambia |
| H-22 | Observación | El 500 de `BusinessException` sale con `"message": null` (se construye solo con la causa); el detalle queda en `devInfo` | Sección 7.7 | Patrón heredado de las Fases 1 y 2; no se cambia en esta fase |

## 10. Matriz de trazabilidad y auditoría

Clasificación: **IV** = implementado y verificado · **INV** = implementado y no verificado · **P** = parcial · **D** = con defectos · **PE** = pendiente.

| Requisito (fuente) | Archivo y método | Prueba | Resultado | Clasif. |
|---|---|---|---|---|
| El sistema de carga envía contraseña y número de orden (consigna 4.1) | `OrderLoaderRestController.activate`, `ActivationRequest` | Postman "Activación correcta" | 200 | IV |
| Se valida que la orden exista (consigna 4.1) | `OrderBusiness.activate` → `load` | "Activación de orden inexistente" | 404 | IV |
| Se valida el estado esperado (consigna 4.1) | `OrderBusiness.activate` | "Activación de orden en estado 1"; estado 3 simulado (7.6) | 409 | IV |
| Se valida la contraseña (consigna 4.1) | `OrderBusiness.activate` | "Activación con contraseña incorrecta"; auditoría D1 | 403, sin activar | IV |
| Se devuelve el preset (consigna 4.1) | `ActivationResponse`, controller | "Activación correcta" | `preset` 15000 | IV |
| La orden sigue en estado 2 (consigna 4.1) | `OrderBusiness.activate` (no toca `status`) | "Consultar la orden activada" | estado 2 | IV |
| Lectura con masa, densidad, temperatura, caudal y estampa (consigna 4.2) | `DetailRequest`, `OrderDetail` | Lecturas válidas 1 y 2 | guardadas | IV |
| Descartar caudal ≤ 0 (consigna 4.2) | `OrderDetailValidator.discardReasons` | Postman caudal 0 y −5; test unitario | `saved: false` | IV |
| Descartar masa ≤ 0 (consigna 4.2) | ídem | Postman masa 0; test unitario (0 y negativa) | `saved: false` | IV |
| Descartar masa menor que la anterior (consigna 4.2) | ídem, con `order.getLastAccumulatedMass()` | Postman masa 200 < 250; test unitario | `saved: false` | IV |
| Densidad entre 0 y 1 (consigna 4.2, decisión 8) | ídem | Postman densidad 0, 1, 1.5; test unitario (también −0.2 y `NaN`) | `saved: false` | IV |
| Cabecera con el último valor válido (consigna 4.2) | `OrderDetailBusiness.add` | "Consultar la cabecera..." (3 veces) | últimos valores correctos | IV |
| Detalle con el histórico (consigna 4.2) | `OrderDetail`, `OrderDetailRepository` | `SELECT COUNT(*)` = 4 | solo las válidas | IV |
| Inicio de carga = primer registro válido (consigna 4.2) | `OrderDetailBusiness.add` | Cabecera tras lecturas y tras casos límite | no cambia | IV |
| Fin de carga = último registro válido (consigna 4.2, D8) | ídem | Cabecera tras dos lecturas y tras la cuarta lectura | actualizado | IV |

| Requisito (fuente) | Archivo y método | Prueba | Resultado | Clasif. |
|---|---|---|---|---|
| Guardar todas las lecturas válidas (consigna 4.2, opcional) | ídem | 100 lecturas → 100 filas | sí | IV |
| Datos de detalle solo en estado 2 (consigna, plan §2) | `OrderDetailBusiness.add` (409) | Postman estado 1; estado 3 simulado | 409 | IV |
| Lecturas solo con la orden activada (plan, `CLAUDE.md` 8.1) | ídem | "Lectura de orden no activada" | 409 `activada=false` | IV |
| `activationDateTime` (decisión 7) | `Order`, `OrderBusiness.activate` | "Consultar la orden activada" | presente | IV |
| D1 contraseña incorrecta → 403 | `ForbiddenException`, `activate`, controller | Postman 403; auditoría | 403 sin contraseña en el cuerpo | IV |
| D2 idempotencia | `activate` (`if activationDateTime == null`) | "Activar de nuevo" + consulta | misma fecha | IV |
| D3 respuesta `{saved, reason}` siempre 200 | `DetailResult`, `DetailResponse` | todas las lecturas | 200 | IV |
| D4 campo ausente → 400 | `DetailRequest` con `Double`; `add` | "Lectura sin un campo" | 400 | IV |
| D5 masa igual válida, anterior = cabecera | `OrderDetailValidator` (`<`), `add` | "masa igual a la anterior"; test unitario | `saved: true` | IV |
| D6 estampa = hora de recepción | `add` (`LocalDateTime.now()`) | Lectura con `timestamp` de 2020 + cabecera | se ignora el del cliente | IV |
| D7 orden de comprobaciones | `activate`, `add` | Auditoría 7.5 (casos combinados) | según lo decidido | IV |
| D8 sin campo duplicado | `Order` | Revisión del modelo | solo `loadingEndDateTime` | IV |
| D9 sin historial en la activación | `activate` | "Historial de estados tras activar" | 2 registros | IV |
| D10 `IOrderDetailBusiness` aparte | `IOrderDetailBusiness`, `OrderDetailBusiness` | Revisión de código | sí | IV |
| Atomicidad detalle + cabecera (criterio 12) | `add` con `@Transactional(rollbackFor = Exception.class)` y `saveAndFlush` | `CHECK` temporal (7.7) | todo revertido, `StandartResponse` 500 | IV |
| Errores con `StandartResponse` también ante fallas de base | `activate` (`saveAndFlush` desde `4ed54c8`) | *Trigger* temporal (7.7), antes y después del arreglo | Antes: 500 con formato de Spring (H-14). Después: 500 con `StandartResponse` | IV (H-14 corregido) |
| Contraseña oculta en consultas, mensajes y logs | `Order` (`@JsonIgnore`), mensajes fijos | Postman, Swagger, búsqueda en el log | no aparece | IV |
| Swagger de los dos endpoints | `@Operation`, `@ApiResponse` | `/v3/api-docs` | completos | IV |

| Requisito (fuente) | Archivo y método | Prueba | Resultado | Clasif. |
|---|---|---|---|---|
| Postman que pruebe todas las APIs (parcial obligatorio) | Colección, carpeta 03 | newman | 160/160 | IV |
| Eficiencia con ≈ 1 lectura por segundo | `add` (masa anterior desde la cabecera) | 100 lecturas | 79,6 ms promedio | IV |
| Esquema sobre base existente | `ddl-auto=update`, columnas nulables | Arranque sobre base de la Fase 1 | sin errores | IV |
| Lista de detalles fuera de `Order` | `Order`, `OrderDetail` (`LAZY`, `@JsonIgnore`) | Postman "sin lista de detalles" | sí | IV |
| Nombres de campos del JSON iguales a la consigna original | DTO | — | sin el PDF | **INV** (no verificado contra el original) |
| `GET /orders/{n}/details` paginado (decisión 11) | `OrderRestController.details`, `OrderDetailBusiness.list`, `OrderDetailRepository.findByOrder` | Postman "Lecturas guardadas..." (6 pedidos); Swagger | 200 con páginas por `id` ascendente, tope 100, 404, 401 | IV |
| Frecuencia de almacenamiento configurable (opcional del parcial) | — | — | se guardan todas | PE (opcional, fuera de alcance) |

**Veredicto de la auditoría:** la Fase 3 está **completa**. Los 19 criterios están cumplidos y verificados; ningún criterio esencial queda sin verificar. El único defecto encontrado (H-14) se corrigió y se verificó, y la decisión 11 se implementó y probó. Queda sin hacer la comparación con el PDF original, que no está disponible.

## 11. Pendientes

| Pendiente | Quién y cuándo |
|---|---|
| Avisar a Martín González del arreglo de H-14 en su archivo (`OrderBusiness.activate`) y de la aprobación de la decisión 11 (la división preveía que la aprobara él) | Tomás Calvo |
| ~~Comparar los nombres del JSON y las reglas con la consigna original en PDF~~ | Hecho en la auditoría: el PDF está en `docs/pdf/consigna-trabajo-integrador-2026.pdf` |
| ~~PR a `main`~~ | Hecho: PR #7 mergeado. Los hallazgos de la auditoría (F-01, F-02, F-13) se corrigen en un PR posterior |
| Fase 4: cerrar la orden (2 → 3) | Rama `fase-4-cierre-orden`, con autorización |

# Parte 2 — Guía de estudio de la Fase 3

## A. Objetivo de la fase

El Punto 3 conecta el **sistema de carga** con el backend:

1. Que solo pueda cargar quien tiene la contraseña que generó la balanza (activación).
2. Que el sistema de carga sepa **cuánto cargar** (el preset).
3. Recibir los datos del caudalímetro en tiempo real, **descartar** los inválidos y guardar los válidos.

4. Dejar en la **cabecera** de la orden un resumen siempre actualizado (últimos valores, inicio y fin de carga), que usarán la Fase 5 (producto cargado = última masa) y el frontend del final (monitoreo, tiempo transcurrido).

## B. Conceptos teóricos

### 1. Transacciones y excepciones *checked* (clase 3 de la cátedra)

Igual que en la Fase 2: las excepciones de negocio son *checked* y Spring solo revierte ante `RuntimeException`, así que se declara `@Transactional(rollbackFor = Exception.class)`. En las lecturas, el detalle y la cabecera están en la misma transacción.

### 2. Cuándo escribe JPA: `save` frente a `saveAndFlush` (complementario)

Dentro de una transacción, `save` sobre una entidad ya cargada solo la marca como modificada: el `UPDATE` se envía al **confirmar** (*flush* en el *commit*), es decir, **después** de que el método terminó. Si la base falla ahí, el `catch` del método ya no está y la excepción sale por fuera del controller (500 con el formato por defecto de Spring). `saveAndFlush` fuerza el envío en ese momento, **dentro** del `try`. Por eso las lecturas usan `saveAndFlush` (Parte 2) y, desde el arreglo de H-14, también la activación.

### 3. Códigos HTTP (complementario: la cátedra usa 404, 302 y 500)

- **400**: el pedido está mal formado o incompleto.
- **401**: no hay token o es inválido (lo resuelve el filtro JWT).
- **403**: el token es válido pero la operación no está permitida (contraseña incorrecta).
- **404**: la orden no existe.
- **409**: la orden existe pero su estado no permite la operación.
- **200 con `saved: false`**: el pedido es correcto y se procesó; el dato se descartó por regla de negocio.

### 4. Idempotencia (complementario)

Una operación es idempotente si repetirla deja el sistema igual que hacerla una vez. La activación lo es: si el sistema de carga reintenta, recibe el mismo preset y `activationDateTime` no cambia.

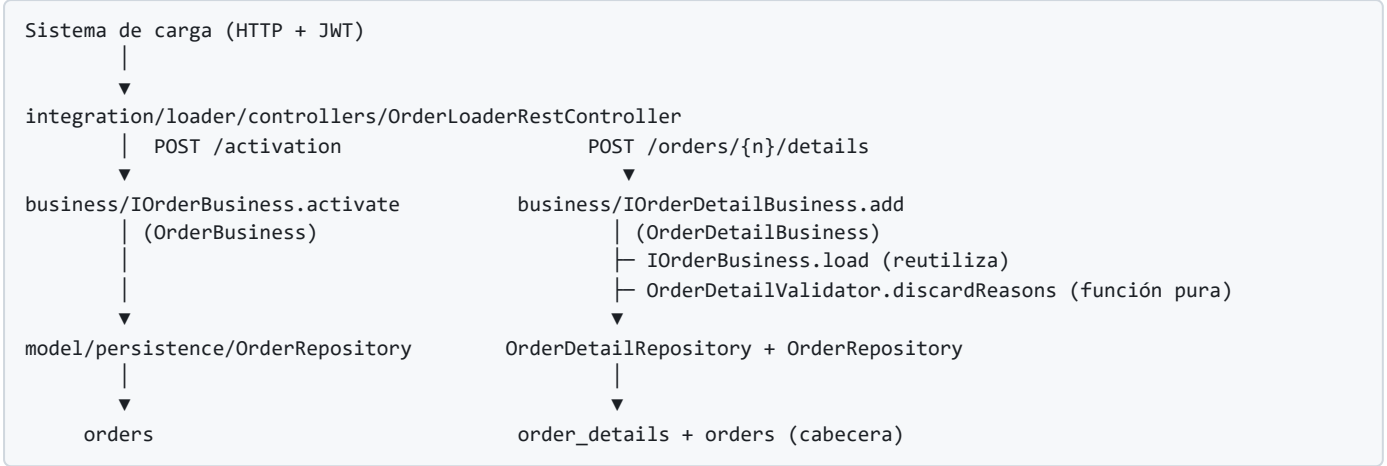
### 5. Relaciones JPA y serialización (clases 2 y 7)

`OrderDetail` → `Order` es `@ManyToOne(fetch = LAZY)` con `@JsonIgnore`: el detalle no carga ni publica la orden. No hay `@OneToMany` inverso en `Order`: con una lectura por segundo, `GET /orders/{n}` traería miles de filas.

### 6. Tipos objeto en los DTO (complementario)

Con `double`, un campo ausente llega como `0.0` y se confundiría con un valor inválido (descarte). Con `Double` llega como `null` y se distingue: ausente = 400, inválido = descarte.

## C. Arquitectura




- **Una interfaz de negocio por entidad** (D10), como la cátedra: `IOrderBusiness` para la orden y `IOrderDetailBusiness` para las lecturas.
- **DTO por sistema externo:** `integration/loader/model` desacopla el contrato del sistema de carga de las entidades.
- **Función pura para las reglas:** `OrderDetailValidator` no depende de Spring ni de la base, por eso se prueba con JUnit sin levantar la aplicación.

## D. Código paso a paso

### 1. La activación (`OrderBusiness.activate`)

```
@Override
@Transactional(rollbackFor = Exception.class)
public Order activate(Integer orderNumber, Integer password)
        throws NotFoundException, BadRequestException, ConflictException, ForbiddenException, BusinessException {
    // Las comprobaciones van fuera del try: si no, el catch las convertiria en 500
    if (orderNumber == null || password == null) {
        throw BadRequestException.builder().message("orderNumber y password son obligatorios").build();
    }
    Order order = load(orderNumber);
    if (order.getStatus() != OrderStatus.INITIAL_WEIGHING_REGISTERED) {
        throw ConflictException.builder().message("La orden " + orderNumber
                + " no está lista para activar (estado " + order.getStatus().getCode() + ")").build();
    }
    // El mensaje no repite la contrasena recibida ni la guardada
    if (!password.equals(order.getActivationPassword())) {
        throw ForbiddenException.builder().message("La contraseña de activación es incorrecta").build();
    }
    // Idempotente: si ya estaba activada se devuelve el preset sin tocar activationDateTime
    if (order.getActivationDateTime() == null) {
        try {
            order.setActivationDateTime(LocalDateTime.now());
            // saveAndFlush: un error de base sale aca, dentro del try, y no al confirmar (H-14)
            order = orderDAO.saveAndFlush(order);
        } catch (Exception e) {
            log.error(e.getMessage(), e);
            throw BusinessException.builder().ex(e).build();
        }
    }
    return order;
}
```

*Detalle:* `password.equals(...)` compara `Integer` por valor (con `==` se compararían referencias).

## 2. Las reglas de descarte (`OrderDetailValidator`)

```
static List<String> discardReasons(double accumulatedMass, double density, double flowRate, Double previousMass)
{
    List<String> reasons = new ArrayList<>();
    if (!(flowRate > 0)) reasons.add("el caudal (flowRate) debe ser mayor que 0");
    if (!(accumulatedMass > 0)) reasons.add("la masa acumulada (accumulatedMass) debe ser mayor que 0");
    if (previousMass != null && accumulatedMass < previousMass)
        reasons.add("la masa acumulada es menor que la anterior (" + previousMass + ")");
    if (!(density > 0 && density < 1)) reasons.add("la densidad (density) debe estar entre 0 y 1, sin
incluirlos");
    return reasons;
}
```

*Detalle:* `!(x > 0)` es verdadero para `NaN` (cualquier comparación con `NaN` es falsa); `x <= 0` lo dejaría pasar. Se usa `<` y no `<=` para que una masa igual sea válida (D5). El código real tiene las mismas condiciones con llaves.

## 3. La recepción de lecturas (`OrderDetailBusiness.add`, parte central)

```
List<String> reasons = OrderDetailValidator.discardReasons(accumulatedMass, density, flowRate,
        order.getLastAccumulatedMass());
if (!reasons.isEmpty()) {
    return DetailResult.discarded(String.join("; ", reasons));
}
try {
    LocalDateTime estampa = LocalDateTime.now(); // hora de recepción, no la del cliente
    OrderDetail detail = new OrderDetail(null, order, accumulatedMass, density, temperature, flowRate, estampa);
    orderDetailDAO.save(detail);
    order.setLastAccumulatedMass(accumulatedMass);
    order.setLastDensity(density);
    order.setLastTemperature(temperature);
    order.setLastFlowRate(flowRate);
    if (order.getLoadingStartDateTime() == null) {
        order.setLoadingStartDateTime(estampa);
    }
    order.setLoadingEndDateTime(estampa);
    orderDAO.saveAndFlush(order);
    return DetailResult.saved();
} catch (Exception e) {
    log.error(e.getMessage(), e);
    throw BusinessException.builder().ex(e).build();
}
```

*Detalle:* los `Double` se pasan al validador que recibe `double` (*unboxing*); es seguro porque antes se comprobó que no son nulos.

## 4. El endpoint de lecturas (`OrderLoaderRestController.addDetail`)

```
@PostMapping(value = "/orders/{orderNumber}/details", consumes = MediaType.APPLICATION_JSON_VALUE,
        produces = MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<?> addDetail(@PathVariable Integer orderNumber, @RequestBody DetailRequest body) {
    try {
        DetailResult result = orderDetailBusiness.add(orderNumber, body.getAccumulatedMass(),
                body.getDensity(), body.getTemperature(), body.getFlowRate());
        return new ResponseEntity<>(new DetailResponse(result.isSaved(), result.getReason()), HttpStatus.OK);
    } catch (BadRequestException e) { ... 400 ... }
    catch (NotFoundException e) { ... 404 ... }
    catch (ConflictException e) { ... 409 ... }
    catch (BusinessException e) { ... 500 ... }
}
```

## E. Trazabilidad de los datos

Orden de la corrida de Postman `120288201` (`id = 6`):

1. **Antes:** estado 2, `tare = 14500`, `activation_password` guardada, los 7 campos de carga en `NULL`, 0 filas en `order_details`.
2. **Activación** `{"orderNumber": 120288201, "password": <contraseña>}` → `UPDATE orders SET activation_date_time = '2026-10-27 13:58:09.810684'`. El estado sigue en 2 y `order_status_logs` sigue con 2 filas.
3. **Lectura 1** (100 kg, `2026-10-28T10:00:01`) → `INSERT order_details (100, 0.5, 20, 3000, '2026-10-28 10:00:01')` y `UPDATE orders` con `last_* = 100 / 0.5 / 20 / 3000`, `loading_start_date_time = loading_end_date_time = '2026-10-28 10:00:01'`.
4. **Lectura 2** (250 kg, `10:00:06`) → otra fila; la cabecera pasa a `250 / 0.52 / 19.5 / 3600` y el fin a `10:00:06`. El inicio no cambia.
5. **7 descartes** → ningún `INSERT` ni `UPDATE`.
6. **Masa igual (250)** y cuarta lectura **(400)** → 2 filas más; la cabecera termina en `400 / 0.54 / 18.5 / 3400` y el fin con la hora del servidor (`2026-10-27 13:58:11.927593`).
7. **Resultado:** `SELECT COUNT(*) FROM order_details WHERE id_order = 6` → **4**.

## F. Ejemplo práctico de punta a punta

```
TOKEN=$(curl -s -X POST http://localhost:8080/api/v1/login -d "username=admin&password=admin123")

# 1. Activar (la contraseña la devolvió la tara: {"activationPassword": 36797})
curl -i -X POST http://localhost:8080/api/v1/integration/loader/activation \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"orderNumber": 120503639, "password": 36797}'
# HTTP/1.1 200   { "preset" : 15000.0 }

# 2. Lectura válida
curl -s -X POST http://localhost:8080/api/v1/integration/loader/orders/120503639/details \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"accumulatedMass": 250.0, "density": 0.52, "temperature": 19.5, "flowRate": 3600.0}'
# { "saved" : true, "reason" : null }

# 3. Lectura descartada (masa menor que la anterior)
curl -s -X POST http://localhost:8080/api/v1/integration/loader/orders/120503639/details \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"accumulatedMass": 200.0, "density": 0.52, "temperature": 19.5, "flowRate": 3600.0}'
# { "saved" : false, "reason" : "la masa acumulada es menor que la anterior (250.0)" }

# 4. Cabecera
curl -s http://localhost:8080/api/v1/orders/120503639 -H "Authorization: Bearer $TOKEN"
```

Los pasos 1 a 3 son equivalentes a pedidos de la carpeta 03 (segunda corrida de newman, orden `120503639`). Respuesta del paso 4 al final de esa corrida (fragmento real, sin camión, chofer, cliente ni producto):

```
{
  "orderNumber": 120503639,
  "status": 2,
  "tare": 14500,
  "activationDateTime": "2026-10-27T14:01:45.532073",
  "loadingStartDateTime": "2026-10-28T10:00:01",
  "loadingEndDateTime": "2026-10-27T14:01:47.933983",
  "lastAccumulatedMass": 400,
  "lastDensity": 0.54,
  "lastTemperature": 18.5,
  "lastFlowRate": 3400
}
```

*(No aparece `activationPassword` ni ninguna lista de detalles. El fin quedó antes que el inicio por H-20, ya corregido: hoy la estampa siempre es la hora del servidor.)*

Contraseña incorrecta:

```
HTTP/1.1 403 Forbidden
{ "message" : "La contraseña de activación es incorrecta", "code" : 403, "devInfo" : "..." }
```

## G. Preguntas del profesor para la defensa oral

1. ¿Por qué la contraseña incorrecta responde 403 y no 401?
2. ¿Qué significa que la activación sea idempotente y por qué se eligió así?
3. ¿Por qué una lectura descartada responde 200 y no 400?
4. ¿Por qué una densidad de 1.5 se descarta pero una lectura sin `density` da 400?
5. ¿Por qué los DTO usan `Double` e `Integer` en lugar de `double` e `int`?
6. ¿Cuál es "la masa anterior" con la que se compara y por qué se toma de la cabecera?
7. ¿Por qué se escribe `!(flowRate > 0)` en lugar de `flowRate <= 0`?
8. ¿En qué orden se hacen las comprobaciones de las lecturas y por qué ninguna está dentro del `try`?
9. ¿Cómo se garantiza que el detalle y la cabecera se guarden juntos o ninguno? ¿Cómo lo verificaron?
10. ¿Por qué las lecturas usan `saveAndFlush` y qué pasa en la activación, que usa `save`?
11. ¿Por qué `Order` no tiene una lista de `OrderDetail`?
12. ¿Por qué la activación no escribe en el historial de estados?
13. Cuando la Fase 4 pase la orden a estado 3, ¿qué hay que cambiar para que se rechacen las lecturas?
14. ¿Qué limitaciones conocidas tiene la fase?
15. ¿Cómo se comprobó en Postman que la contraseña no se filtra?
16. ¿Cómo funciona `GET /orders/{n}/details` y por qué ignora el `sort` del cliente y limita el tamaño de página?

## H. Respuestas modelo

1. **403 y no 401.** El 401 significa "no sé quién sos": falta el token o es inválido, y lo resuelve el filtro JWT. Si el token es válido y la contraseña de activación no coincide, el usuario está identificado pero la operación no está permitida: eso es 403. Con 401, el sistema de carga podría creer que tiene que volver a loguearse (D1).
2. **Idempotencia.** Activar dos veces con la contraseña correcta deja todo igual que activar una vez: devuelve el mismo preset y no cambia `activationDateTime`, porque solo se guarda si es nulo. Si el sistema de carga pierde la respuesta y reintenta, no queda bloqueado (D2).
3. **Descarte con 200.** El pedido es correcto y el backend lo procesó: aplicó la regla de la consigna y decidió no guardar el dato. No es un error del cliente; el caudalímetro sigue mandando lecturas. La diferencia está en `saved` (D3).

4. **Densidad inválida frente a ausente.** 1.5 es un valor físico informado que no cumple la regla `0 < d < 1`: se descarta como las otras reglas de la consigna. Un campo ausente es un pedido incompleto, un error de formato del cliente: 400 (D4).
5. **Tipos objeto.** Con `double`, Jackson pone `0.0` si el campo falta, y esa lectura se descartaría por "masa ≤ 0" en lugar de dar 400. Con `Double` llega `null` y se distingue "ausente" de "inválido". En la activación, `Integer password` permite dar 400 si falta.
6. **La masa anterior.** Es la última masa **válida**, guardada en la cabecera (`lastAccumulatedMass`). Si se tomara la última recibida, una lectura descartada (por ejemplo, 9999 por un pico) haría descartar todas las siguientes. Además evita recorrer el histórico en cada lectura (eficiencia, D5).
7. **`!(x > 0)`.** Toda comparación con `NaN` da falso. `NaN <= 0` es falso y la lectura pasaría; `!(NaN > 0)` es verdadero y se descarta.
8. **Orden de las lecturas.** 404 (la orden existe) → 400 (campos presentes) → 409 (estado 2 y activada) → descarte. Van fuera del `try` porque el `catch (Exception)` las convertiría en `BusinessException` y el cliente recibiría 500 en vez del código correcto (D7).
9. **Atomicidad.** `add` es `@Transactional(rollbackFor = Exception.class)`: el `INSERT` del detalle y el `UPDATE` de la cabecera están en la misma transacción. Se verificó con una restricción `CHECK` temporal que hacía fallar solo el `UPDATE`: respondió 500 y no quedó ni el detalle ni el cambio de cabecera.
10. **`saveAndFlush`.** `save` deja el `UPDATE` para el *commit*, que ocurre después del `try`; un error de base ahí sale con el formato por defecto de Spring. `saveAndFlush` lo envía dentro del `try` y el error se traduce a `StandartResponse`. La activación usaba `save`: con un *trigger* se comprobó que su 500 no era `StandartResponse` (H-14). Se cambió a `saveAndFlush` y la misma prueba ahora responde con `StandartResponse`.
11. **Sin lista de detalles.** Con una lectura por segundo, una carga de una hora son 3600 filas: `GET /orders/{n}` las traería todas. El detalle apunta a la orden (`@ManyToOne LAZY`) y no al revés.
12. **Activación sin historial.** El historial registra cambios de estado y la activación no cambia el estado (la orden sigue en 2). Para saber si está activada está `activationDateTime` (decisión 7, D9).
13. **Fase 4.** Nada de la Fase 3: la comprobación es "estado 2 **y** activada". En estado 3 la lectura responde 409. Se verificó cambiando el estado a 3 en la base local.
14. **Limitaciones.** Activaciones simultáneas (H-16; las lecturas simultáneas, H-15, se corrigieron), contraseña de 5 dígitos sin límite de intentos (H-17), token de 1 h (H-18), y números aceptados como texto (H-21). El único defecto (H-14) ya está corregido.
15. **Contraseña en Postman.** En el pedido del 403, la aserción revisa que el cuerpo completo no contenga ni la contraseña enviada (`wrongPasswordLoad`) ni la correcta (`activationPasswordLoad`). En "Consultar la orden activada" se revisa que `activationPassword` no exista en el JSON.
16. **Consulta del histórico.** `OrderRestController.details` recibe `page` y `size` (20 por defecto), acota `size` a 100 y arma un `PageRequest` **sin** el `sort` del cliente; `OrderDetailBusiness.list` busca la orden (404 si no existe, fuera del `try`) y el repositorio trae la página con `order by d.id asc` fijo en la consulta. Se responde `PagedModel` (`content` + `page`), igual que `GET /orders`. El orden fijo hace que la página sea predecible (de la lectura más antigua a la más reciente) y el tope de 100 evita que un cliente pida miles de filas de una sola vez (decisión 11).

## I. Errores frecuentes

1. **Usar `double` en los DTO:** un campo ausente se descarta en vez de dar 400.
2. **Poner las comprobaciones dentro del `try`:** 404, 400, 409 o 403 salen como 500.
3. **Olvidar `rollbackFor = Exception.class`:** quedaría el detalle sin la cabecera (o al revés).
4. **Lanzar una excepción para descartar:** descartar no es un error; se devuelve `DetailResult.discarded(...)` sin escribir nada.
5. **Comparar con la última masa recibida** en lugar de la última válida.

6. **Usar `x <= 0`:** deja pasar `NaN`.
7. **Agregar `@OneToMany` en `Order`:** respuestas enormes y riesgo de serialización circular.
8. **Repetir la contraseña en el mensaje de error o loguearla.**
9. **Comparar `Integer` con `==`:** compara referencias; se usa `equals`.
10. **Usar `orderNumber` o `activationPassword` en la carpeta 03 de Postman:** pisa las variables de las Fases 1 y 2.
11. **Mandar un `timestamp` (con o sin zona):** se ignora; la estampa es la hora de recepción del servidor (antes H-19).

## J. Lista de comprobación para rendir

- [ ] Rutas y cuerpos: `POST /api/v1/integration/loader/activation` y `POST /api/v1/integration/loader/orders/{orderNumber}/details`.
- [ ] Los códigos de cada endpoint y el orden de las comprobaciones (D7).
- [ ] Las cuatro reglas de descarte, por qué la masa igual es válida y por qué la temperatura no tiene regla.
- [ ] Qué guarda la cabecera y qué guarda el detalle; inicio y fin de carga.
- [ ] Por qué 403, por qué idempotente, por qué 200 al descartar.
- [ ] `save` frente a `saveAndFlush` y el hallazgo H-14.
- [ ] Cómo se probó la atomicidad y la preparación para la Fase 4.
- [ ] `GET /api/v1/orders/{orderNumber}/details?page=&size=`: orden fijo, tope de 100 y `PagedModel`.
- [ ] Mostrar la carpeta 03 de Postman y newman con 160 aserciones y 0 fallas.
- [ ] `SELECT COUNT(*) FROM order_details WHERE id_order = <id>` para mostrar que los descartes no se guardan.

### Qué debe poder explicar cada integrante

La evaluación es individual: cada uno tiene que poder explicar **toda** la fase, pero con más detalle su parte.

| Integrante | Su parte | Lo que tiene que dominar |
|---|---|---|
| **Martín González** (Parte 1) | Modelo y activación | Los 7 campos de `Order` y por qué son nulables; `OrderDetail` (`LAZY`, `@JsonIgnore`, `reading_timestamp`) y por qué no hay `@OneToMany`; `ForbiddenException` y el 403; `activate` paso a paso (orden 400 → 404 → 409 → 403, idempotencia, sin historial); los DTO con `Integer`; el hallazgo H-14 y por qué `saveAndFlush` lo corrige (el arreglo quedó en su archivo) |
| **Joaquín Contreras** (Parte 2) | Lecturas y descarte | `OrderDetailValidator` (las 4 reglas, `!(x > 0)` y `NaN`, masa igual válida); sus 8 tests; `OrderDetailBusiness.add` (orden 404 → 400 → 409 → descarte, masa anterior desde la cabecera, estampa del servidor); `DetailResult` y la respuesta 200; `saveAndFlush` y la prueba de falla; la medición de eficiencia |
| **Tomás Calvo** (Parte 3) | Pruebas, auditoría y documentación | La carpeta 03 de Postman (variables propias, aserciones con `include`, timestamps fijos); newman y `mvnw test`; la matriz de auditoría; las pruebas de la Fase 4 (estado 3), atomicidad (`CHECK`) y activación (*trigger*); los hallazgos H-14 a H-22 y el arreglo de H-14; la consulta paginada `GET /orders/{n}/details` (decisión 11) |
| **Benjamín Alasino** (revisión) | Revisión del PR | El recorrido completo de las dos peticiones (sección 4) y las decisiones D1 a D10 |